

Actividad Obligatoria No. 1

Evaluación de sistema de chat concurrente por consola en Python mediante métricas estáticas

Micaela Palau, Federico Pucci, Rocío Chauque, Micaela Scinardo Ratto, Alejandro Garay

Universidad Nacional de La Matanza

Buenos Aires, Argentina

{mpalau@alumno.unlam.edu.ar, fpucci@alumno.unlam.edu.ar, rchauque@alumno.unlam.edu.ar, miscinardoratto@alumno.unlam.edu.ar, agaray645@alumno.unlam.edu.ar}

Resumen

Este trabajo evalúa dos implementaciones de un chat concurrente por consola en Python generadas con herramientas de *vibecoding* —Gemini y Sonet— a partir de un prompt técnico. El objetivo es analizar si el código cumple los requisitos de concurrencia (múltiples clientes, comunicación sin turnos, límite de 280 caracteres, cierre por palabra clave) y cuantificar con métricas el código generado.

1. Introducción

Los sistemas concurrentes requieren coordinar múltiples actividades que comparten recursos y se ejecutan “al mismo tiempo”. En el contexto de **Python**, la concurrencia basada en **hilos** (módulo `threading`) resulta adecuada para cargas **I/O-bound** —como la comunicación por **sockets**—, mientras que la basada en procesos aborda trabajos **CPU-bound**. Desde la perspectiva de calidad, la concurrencia introduce desafíos adicionales: **sincronización**, **delimitación de mensajes** (framing), **manejo de errores** y **cierres ordenados** para evitar bloqueos, cuelgues o estados inconsistentes.

El objetivo de este trabajo es doble. Primero, verificar si dos implementaciones de chat por consola generadas con herramientas de *vibecoding* (Gemini y Sonet) satisfacen los requisitos funcionales de la consigna. Segundo, evaluar cuantitativamente el código con métricas.

Para ello adoptamos dos métricas ampliamente difundidas y disponibles en herramientas libres: (i) la Complejidad Ciclomática (CC), que estima el número de caminos lógicos independientes de una función o método (a mayor CC, aumenta el esfuerzo de prueba y el riesgo de defectos), y (ii) el Índice de Mantenibilidad (MI).

El alcance del análisis sigue el siguiente recorrido: (1) se presentan las herramientas de *vibecoding* seleccionadas y el prompt técnico que guió la generación; (2) se describen las implementaciones resultantes y se realiza un análisis crítico de sus implementaciones (modelo de hilos, sincronización, tratamiento de E/S); (3) se aplican las métricas (Radon CC y MI) y se presentan los hallazgos cuantitativos.

En síntesis, el trabajo busca articular criterios teóricos de concurrencia con evidencia empírica provista por métricas y, a la vez, valorar críticamente el uso de herramientas de IA en el desarrollo concurrente.

2. Desarrollo

2.1. Herramientas de vibecoding seleccionadas

Para esta evaluación se seleccionaron dos herramientas de *vibecoding* con perfiles complementarios: **Gemini 2.5 Flash**, un modelo orientado a baja latencia y respuesta rápida, útil para idear soluciones, generar esqueletos de código y recibir sugerencias contextuales “al vuelo” (especialmente práctico en iteraciones cortas y prototipado); y **Sonet 4.5 integrado a Visual Studio Code**, enfocado en el flujo de desarrollo dentro del IDE con asistencia en línea (autocompletado, refactorización guiada, explicación de fragmentos, generación de pruebas y comandos tipo “copilot”), lo que facilita tener a mano el código, la terminal y el chat en un mismo entorno.

2.2. Prompt técnico utilizado

De los diferentes tipos de prompts que se conocen en la literatura, se ha elegido uno basado en el tipo Prompt

Patterns, subtipo Input Semantics. La característica distintiva es que permite observar una estructura lógica, clara y coherente con la metodología utilizada Ingeniería de Requerimientos.

La estructura que sigue este prompt técnico es:

- LENGUAJE DE PROGRAMACIÓN
- PLATAFORMA
- FUNCIONALIDADES A IMPLEMENTAR
- REQUISITOS ESPECÍFICOS
- RESTRICCIONES

Debe destacarse que el prompt técnico final llevo 2 iteraciones: la primera establecía esperas bloqueantes y no había concurrencia. En la segunda y final que se observa a continuación se agregó la capacidad de manejo de hilos y conversaciones entre clientes sin turnos.

A continuación, puede observarse el prompt técnico final:

LENGUAJE DE PROGRAMACION: Python.

PLATAFORMA: Windows.

FUNCIONALIDADES A IMPLEMENTAR:

- Chat en una computadora utilizando únicamente la consola.

REQUISITOS ESPECÍFICOS

- Programa Servidor:
 - Se encarga de “escuchar” conexiones de clientes.
 - Debe permitir que varios clientes se conecten y participen al mismo tiempo en una misma sala de chat.
 - Una vez que un cliente se conecta, mantiene la comunicación y puede recibir y enviar mensajes sin turnos, sin bloquear al resto de las conversaciones.
 - Al conectarse, recibe el nombre de usuario del cliente y lo utilizará para anteponerlo a los mensajes que reenvíe a los demás participantes.
 - Si el servidor ingresa la palabra clave “chau”, cerrará ordenadamente todas las conexiones y finalizará el programa.
- Programa Cliente:
 - Se conecta al servidor usando la IP y puerto del servidor.
 - Se ingresa el nombre del usuario al inicio para que anteceda a los mensajes que verán los demás.
 - Envía y recibe mensajes sin turnos, pudiendo seguir recibiendo mientras el usuario está escribiendo.
 - Si el cliente envía la palabra clave “chau”, cerrará su conexión de forma

ordenada.

RESTRICCIONES

- Los mensajes no son por turno: ambos extremos deben poder enviar y recibir en paralelo sin que uno dependa del otro.
- El mensaje tiene un límite de 280 caracteres; si supera ese límite, se descarta el resto.
- La IP que usará el servidor es 127.0.0.1 con el puerto 5000.
- El sistema no debe quedar bloqueado si un participante deja de responder; las demás conversaciones deben continuar funcionando con normalidad.

2.3. Código generado por Gemini

El código de Gemini cumple el objetivo funcional y presenta una estructura conocida y efectiva para un chat básico: hilos por conexión, recepción y envío en paralelo, y validaciones mínimas como el límite de 280 caracteres y la palabra clave “chau”. Una debilidad estructural: el broadcast se realiza con E/S bloqueante dentro del lock, esto podría traer como consecuencia que cualquier cliente lento sea un cuello de botella para todos. A esto se suman se le suma la ausencia de framing y timeouts, y una terminación abrupta con `os._exit(0)` que saltea limpieza y coordinación de hilos. Puede concluirse que se trata de un prototipo funcional y directo, útil para demostrar concurrencia básica, pero tendría una mejora sustancial si se hace más robusto en el plano de sincronización, protocolo y cierre para en escenarios reales. Por último, no menos importante, el look-and-feel es amigable utilizando emojis y tonos empáticos con el usuario.

2.4. Código generado por Sonet

La solución de Sonet también pasa la prueba funcional y ordena mejor algunas responsabilidades (por ejemplo, la mención explícita de Lock sobre la colección de clientes y la separación de hilos). Sin embargo, su “sistema no bloqueante” no se sostiene: no hay timeouts y `recv()` es bloqueante; además aparecen condiciones de carrera (p. ej., running sin sincronización y modificación concurrente de listas), hilos daemon que pueden terminar sin completar operaciones, y excepciones genéricas que ocultan fallos. El framing también falta, y el buffer fijo de 1024 bytes puede fragmentar mensajes. En comparación con Gemini, Sonet diagnostica más los riesgos de sincronización (enumera condiciones de carrera y posibles deadlocks), pero igualmente necesita mejoras equivalentes: Event para el ciclo de vida, timeouts, framing claro, envío fuera del lock o con colas/hilos de salida por cliente, y terminación ordenada con `join()` y logging. Respecto a su experiencia de usuario, se lo nota más basado en texto y con una experiencia de usuario más simple y primitiva.

3. Resultados

3.1. Métricas de evaluación

En este trabajo se ha elegido medir una dimensión de la calidad del código generado: **Complejidad ciclomática (CC)** y **mantenibilidad (MI)**. La **CC** mide el número de caminos lógicamente independientes (ramificaciones). A mayor **CC**, más casos de prueba y más riesgo de defectos. El **Índice de Mantenibilidad (MI)** combina **CC**, Halstead y tamaño para ofrecer un indicador agregado de facilidad de mantenimiento. Estas métricas ayudan a detectar “puntos calientes” (funciones que conviene refactorizar antes).

Para evaluar los aspectos anteriores se ha decidido utilizar la herramienta **Radon**:

- **radon cc** se usa para para calcular **complejidad ciclomática** por función (escala A–F).
- **radon mi** se usa para calcular el **Índice de Mantenibilidad** a nivel de archivo.

3.2. Resultados obtenidos en código generado por Gemini

El informe emitido por Radon (con la opción **cc**) calculando la Complejidad Ciclométrica es:

```
ClienteChat_Gemini.py
  F 60:0 enviar_mensajes - B (9)
  F 17:0 recibir_mensajes - B (8)
  F 103:0 iniciar_cliente - A (4)
ServidorChat_Gemini.py
  F 18:0 manejar_cliente - B (7)
  F 145:0 iniciar_servidor - B (7)
  F 82:0 broadcast - B (6)
  F 107:0 entrada_servidor - B (6)

7 blocks (classes, functions, methods) analyzed.
Average complexity: B (6.714285714285714)
```

El informe emitido por Radon (con la opción **mi**) calculando el Índice de Mantenibilidad es:

```
ClienteChat_Gemini.py - A (69.64)
ServidorChat_Gemini.py - A (64.88)
```

Radon reporta **CC promedio B=6.71** en 7 bloques: los “puntos calientes” están en el cliente con *enviar_mensajes* **B(9)** y *recibir_mensajes* **B(8)**, y en el servidor con *manejar_cliente* **B(7)** e *iniciar_servidor* **B(7)** (mientras que *broadcast* y *entrada_servidor* quedan en **B(6)**). Son complejidades moderadas y alineadas con un diseño sencillo; no hay picos en C o peores. En cuanto al **MI** es **A** en ambos módulos (**69.64** cliente; **64.88** servidor), lo que indica buena mantenibilidad global.

3.3. Resultados obtenidos en código generado por Sonet

El informe emitido por Radon (con la opción **cc**) calculando la Complejidad Ciclométrica es:

```
cliente_Sonet.py
  M 97:4 ChatClient.send_messages - C (11)
  M 66:4 ChatClient.receive_messages - B (8)
  C 17:0 ChatClient - B (6)
  M 25:4 ChatClient.connect - A (4)
  M 142:4 ChatClient.disconnect - A (3)
  M 18:4 ChatClient.__init__ - A (1)
servidor_Sonet.py
  M 54:4 ChatServer.accept_connections - B (7)
  M 95:4 ChatServer.handle_client - B (7)
  M 150:4 ChatServer.remove_client - B (6)
  C 16:0 ChatServer - A (5)
  M 133:4 ChatServer.broadcast - A (5)
  M 168:4 ChatServer.server_commands - A (5)
  M 181:4 ChatServer.shutdown - A (5)
  M 25:4 ChatServer.start - A (2)
  M 17:4 ChatServer.__init__ - A (1)

15 blocks (classes, functions, methods) analyzed.
Average complexity: B (5.066666666666667)
```

El informe emitido por Radon (con la opción **mi**) calculando el Índice de Mantenibilidad es:

```
cliente_Sonet.py - A (60.83)
servidor_Sonet.py - A (54.30)
```

Aquí la **CC promedio B=5.07** (15 bloques) es **ligeramente mejor** que la de Gemini, pero aparece un **pico local**: *ChatClient.send_messages* llega a **C(11)**, por encima del máximo de Gemini (**B=9**). El resto se mantiene en A/B (*receive_messages* **B(8)**); en el servidor, *accept_connections* y *handle_client* **B(7)**, y varias operaciones en **A**). En mantenibilidad, ambos archivos quedan en **MI A (60.83** cliente; **54.30** servidor), algo **más bajo** que Gemini—sobre todo en el servidor—lo que sugiere más deuda en estructura/documentación/ramas.

4. Conclusiones

En este trabajo se presentaron los análisis de las herramientas de IA Gemini y Sonet para la generación de código (vibecoding). A partir de lo aprendido en las clases de Programación Concurrente, el uso de las IAs para generación de código y la investigación de métricas y su aplicación, se intenta contribuir no solo a nuestro aprendizaje como estudiantes en el área de la concurrencia sino también avanzar en la capacidad de generar juicio crítico a la hora de producir código propio, leer código ajeno, complementar con IA y todo ello evaluándolo de manera profesional mediante las métricas más convenientes.

Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts (10th ed.)*. Wiley.
- [2] Stallings, W. (2015). *Operating Systems: Internals and Design Principles (8th ed.)*. Pearson.
- [3] Carnuccio, Volker, Tomalino, Adagio, Hirschfeldt, Vera. *Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software* [archivo .pdf entregado por la cátedra] Paper_Conaiisi_2025
- [4] Radon's documentation [online] Available: <https://radon.readthedocs.io/en/latest/commandline.html>
- [5] Forbes E. (2017). *Learning Concurrency in Python*. Packt.